

SlotAssignmentServiceV5 vs SlotAssignmentServiceV6

Corrected Technical Verdict (Evidence-backed)

Date: 2026-03-29

Repo: /Users/arunav/Documents/workspace/claude-code/rate-limiter

0) Scope, Method, and Corrections Applied

Primary source of truth: implementation code + tests.

READMEs are treated as secondary and only trusted when consistent with code.

Files analyzed:

- src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV5.kt
- src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV6.kt
- src/main/kotlin/com/ratelimiter/slot/WindowCounterRefreshScheduler.kt
- src/main/kotlin/com/ratelimiter/repo/WindowSlotCounterRepository.kt
- src/main/kotlin/com/ratelimiter/repo/EventSlotRepository.kt
- src/main/kotlin/com/ratelimiter/repo/SkipPointerRepository.kt
- src/main/resources/application.yaml
- src/test/kotlin/com/ratelimiter/SlotAssignmentServiceV5Test.kt
- src/test/kotlin/com/ratelimiter/SlotAssignmentServiceV6Test.kt
- src/main/kotlin/com/ratelimiter/slot/V5_README.md
- src/main/kotlin/com/ratelimiter/slot/V6_README.md

Important correction context from your feedback:

- 1) V6 should not be scored as better on proximity-to-requestedTime.
- 2) V6 should not be scored as blanket winner on correctness for target workloads
- 3) DB-load scoring must include V6 scheduler MERGE/COUNT cost.
- 4) V5 counter contention must be quantified (not assumed severe).
- 5) Throughput analysis must include background scheduler impact.

Citations use "path:line" format.

A) Executive Summary (Corrected)

Overall best default for the stated workloads (500 TPS near-term + 500k @ 100 TP S long-horizon):

- V5 is the safer default under current settings, unless measurements show RL_WN DW_CT row-update lock waits are a major bottleneck.

Why this correction:

- V6 hot path removes counter UPDATE contention, but adds:
 - one extra per-request COUNT transaction,
 - plus repeated scheduler MERGE/COUNT load across pods.

[src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV6.kt:213-214,
src/main/kotlin/com/ratelimiter/repo/EventSlotRepository.kt:163-170,
src/main/kotlin/com/ratelimiter/slot/WindowCounterRefreshScheduler.kt:34-40,
src/main/kotlin/com/ratelimiter/repo/WindowSlotCounterRepository.kt:233-251]
- Proximity behavior is the same picker in both; V6 occupancy hints are staler b y design.

[src/main/kotlin/com/ratelimiter/slot/WindowPicker.kt:25-45,
src/main/kotlin/com/ratelimiter/repo/WindowSlotCounterRepository.kt:141-143,
src/main/kotlin/com/ratelimiter/slot/WindowCounterRefreshScheduler.kt:34-40]
- Correctness is nuanced:
 - V5: no explicit maxSlots check at claim time.
 - V6: has soft guard check, but it is non-atomic (COUNT then INSERT race).

```
[src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV5.kt:181-183,  
src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV5.kt:218-220,  
src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV6.kt:213-214,  
src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV6.kt:219-221]
```

When recommendation changes:

- If measured V5 counter-row lock waits are high enough to dominate p95/p99 latency, and DB has CPU headroom for V6 scheduler + COUNT load, V6 can be preferred.
 - If scheduler is moved to distributed Quartz singleton (cluster-singleton execution), V6 advisory occupancy staleness increases substantially; V5 becomes more attractive.
- ```
[src/main/resources/application.yaml:153-154,
src/main/kotlin/com/ratelimiter/slot/WindowCounterRefreshScheduler.kt:34-40]
```

## B) Corrected Scorecard

| Parameter                                  | V5 assessment                                                                        | V6 assessment                                                                 | Winner                   | Why winner                                                             | Confidence |
|--------------------------------------------|--------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|--------------------------|------------------------------------------------------------------------|------------|
| 1. Global rate limiting behavior           | Shared by windowStart counter table                                                  | Same shared model + guard check at maxSlots                                   | Tie                      | Both depend on shared WNDW_STRT_TS counters/slot table                 | Med        |
| 2. Proximity to requestedTime              | Same weighted picker, fresher occupancy hints                                        | Same weighted picker, staler advisory occupancy                               | Tie (slight V5)          | Picker identical; V6 staleness can increase mis-picks/re-picks         | Med        |
| 3. Correctness at target workloads         | No claim-time hard max check; practical behavior depends on tuning and concentration | Soft guard before insert, but non-atomic race remains                         | Tie                      | Different failure shapes; neither provides strict atomic max guarantee | Med        |
| 4. Performance at high TPS                 | 3-request-path transactions, possible row-update hotspots                            | 4-request-path transactions + scheduler overhead, fewer hot-path update locks | Tie (workload-dependent) | V6 hot path faster under contention, V5 lighter total machinery        | Low-Med    |
| 5. DB load at high TPS                     | Lower background load; inline update cost                                            | Extra per-request COUNT + scheduler MERGE/COUNT across pods                   | V5                       | Total DB work is higher in V6 per current config                       | Med        |
| 6. Maintainability/operational complexity  | Simpler runtime path, no refresh dependency                                          | Requires refresh health/tuning (`every`, `since`)                             | V5                       | Fewer moving parts                                                     | High       |
| 7. Multi-pod (~24) behavior                | Possible update contention when traffic concentrates                                 | Better hot-path write scalability, but scheduler fan-out load                 | Tie                      | Trade-off between update contention and scheduler overhead             | Med        |
| 8. Quarkus -> Quartz distributed scheduler | Mostly unaffected                                                                    | Staleness increases when refresh becomes singleton                            | V5                       | V6 advisory freshness degrades materially                              | High       |
| 9a. Failure modes                          | Fewer async dependencies                                                             | Additional stale-counter/scheduler-lag mode                                   | V5                       | Scheduler failure logs and continues                                   | High       |
| 9b. Consistency model                      | Near-real-time counter updates                                                       | Eventually consistent counter table                                           | V5                       | Inline vs async reconcile                                              | High       |
| 9c. Retry/rollback behavior                | No rollback path on overshoot                                                        | No rollback; re-pick on guard reject                                          | Tie                      | Both non-rollback designs                                              | High       |
| 9d. Hotspot contention                     | Counter row update hotspot possible                                                  | Hot path avoids counter updates                                               | V6                       | Structural removal of counter write hotspot                            | High       |
| 9e. Scheduler lag resilience               | Not refresh-dependent for occupancy freshness                                        | Sensitive to refresh cadence/lookback                                         | V5                       | Refresh lag directly impacts advisory quality                          | High       |
| 9f. Idempotency under race                 | Strong (UNIQUE eventId + duplicate re-read)                                          | Same                                                                          | Tie                      | Same mechanism                                                         | High       |

Evidence for table-level invariants and idempotency:

- Shared counter keyed by window: src/main/kotlin/com/ratelimiter/db/Tables.kt:7-16
- Event unique idempotency: src/main/kotlin/com/ratelimiter/db/Tables.kt:36
- V5 duplicate recovery: src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV5.kt:212-216
- V6 duplicate recovery: src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV6.kt:243-247

## C) Deep Dive by Evaluation Criterion

### C1) Global rate limiting behavior

What code does:

- RL\_WNDW\_CT is keyed by WNDW\_STRT\_TS only (global counter per window).  
[src/main/kotlin/com/ratelimiter/db/Tables.kt:7-16]
- Both V5 and V6 read occupancy from RL\_WNDW\_CT over [from, to).  
[src/main/kotlin/com/ratelimiter/repo/WindowSlotCounterRepository.kt:196-207]
- V6 additionally checks real slot table count for picked window.  
[src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV6.kt:213-214,  
src/main/kotlin/com/ratelimiter/repo/EventSlotRepository.kt:163-170]

Critical caveat (Hypothesis unless caller guarantees alignment):

- V5/V6 generate windows starting exactly from raw requestedTime; no alignment/rounding is applied in either service or API.  
[src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV5.kt:234-237,  
src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV6.kt:261-264,  
src/main/kotlin/com/ratelimiter/api/SlotAssignmentV2Resource.kt:54-62]
- If requestedTime carries arbitrary millis, distinct request streams may land on different 30s lattices, weakening practical global sharing.

Verdict:

- Architectural intent is global in both.
- Practical global behavior depends on timestamp alignment discipline upstream.

### C2) Proximity of scheduledTime to requestedTime

What code does:

- Both versions use identical weighted random picker:  
weight = max(0, threshold - occupancy) \* proximityWeight.  
[src/main/kotlin/com/ratelimiter/slot/WindowPicker.kt:31-35]
- Both add random jitter within selected window.  
[src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV5.kt:190-192,  
src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV6.kt:219-221]

Key difference:

- V5 occupancy hint updated inline with claim (upsertCounter).  
[src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV5.kt:218-220,  
src/main/kotlin/com/ratelimiter/repo/WindowSlotCounterRepository.kt:257-281]
- V6 occupancy hint is eventually consistent (refresh scheduler).  
[src/main/kotlin/com/ratelimiter/slot/WindowCounterRefreshScheduler.kt:34-40,  
src/main/kotlin/com/ratelimiter/repo/WindowSlotCounterRepository.kt:228-251]

Verdict:

- Not a V6 win. Tie, with slight V5 edge for occupancy freshness.

### C3) Correctness at target workloads (including inaccuracies/worst-case)

Production defaults used:

- window-size=30s, max-slots=900, soft-max-percent=90, default-max-duration=8h, chunk=15m, extension-windows=40, max-extensions-beyond=5.  
[src/main/resources/application.yaml:138-154]

Derived values:

- softMax =  $\text{floor}(900 \cdot 90 / 100) = 810$ .  
[src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV5.kt:69,  
src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV6.kt:81]
- Phase1 windows:  $8h / 30s = 960$ .
- Extension windows:  $40 \cdot 5 = 200$ .
- Total searchable windows in exception message path: 1160.  
[src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV5.kt:90-96,  
src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV6.kt:101-107]

Correctness properties:

- V5 does not perform claim-time hard-cap validation/rollback; comments explicitly state no rollback.  
[src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV5.kt:181-183,  
src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV5.kt:196-199]
- V6 performs soft guard COUNT against maxSlots, then INSERT; non-atomic between the two operations.  
[src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV6.kt:213-221]
- V6 phase threshold can be softMax, but guard check is still maxSlots, not softMax.  
[src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV6.kt:131-132,  
src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV6.kt:214]

Test evidence:

- V6 has explicit tests for soft guard behavior and bounded over-allocation assumption.  
[src/test/kotlin/com/ratelimiter/SlotAssignmentServiceV6Test.kt:782-805,  
src/test/kotlin/com/ratelimiter/SlotAssignmentServiceV6Test.kt:528-575]
- V5 tests validate functional phase behavior and counter consistency but do not establish strict maxSlots non-overshoot proof under realistic high concurrency.  
[src/test/kotlin/com/ratelimiter/SlotAssignmentServiceV5Test.kt:421-468]

Verdict:

- Correctness winner should be Tie for these workloads.
- V6 has an explicit guard, but not atomic strictness.
- V5 may be practically safe with tuned headroom + broad distribution, but strict impossibility of overshoot is not provable from current code.

### C4) Performance at high TPS

Request-path transaction patterns (happy path):

- V5: skip read + occupancy read + claim tx (insert + upsert) = ~3 tx.  
[src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV5.kt:73,  
src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV5.kt:117,  
src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV5.kt:207-220]
- V6: skip read + occupancy read + soft-guard COUNT + insert tx = ~4 tx.  
[src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV6.kt:85,  
src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV6.kt:127,  
src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV6.kt:213,  
src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV6.kt:238-241]

At 500 TPS:

- V5 request-path tx/sec ~ 1500.
- V6 request-path tx/sec ~ 2000 (+33%).

Verdict:

- Hot-path micro-performance may favor V6 under counter-row contention.
- End-to-end service performance is workload-dependent once scheduler is included.

---

## C5) DB load at high TPS (explicit 500 TPS + 20/24 pod analysis)

---

V6 adds two DB load sources beyond V5:

- 1) Per-request COUNT on slot table (soft guard).  
[src/main/kotlin/com/ratelimiter/repo/EventSlotRepository.kt:163-170]
- 2) Scheduler MERGE/COUNT query every 3s per pod.  
[src/main/kotlin/com/ratelimiter/slot/WindowCounterRefreshScheduler.kt:34-40,  
src/main/resources/application.yaml:153-154,  
src/main/kotlin/com/ratelimiter/repo/WindowSlotCounterRepository.kt:233-251]

Repository comment gives per-refresh estimate at 500 TPS:

- ~45k to 90k rows counted, 50-100 MERGE upserts/run.  
[src/main/kotlin/com/ratelimiter/repo/WindowSlotCounterRepository.kt:222-225]

For 24 pods, every 3s => 8 refresh runs/sec cluster-wide:

- Scheduler counted rows/sec ~ 360k to 720k.
- Scheduler merge rows/sec ~ 400 to 800.

For 20 pods, every 3s => 6.67 runs/sec:

- Scheduler counted rows/sec ~ 300k to 600k.
- Scheduler merge rows/sec ~ 333 to 667.

Plus V6 guard COUNTs:

- 500 COUNTs/sec in hot path (before retries/re-picks).

Verdict:

- Under per-JVM scheduler, total DB work in V6 is higher than V5 for this workload.
- DB-load winner: V5.

---

## C6) Maintainability / operational complexity

---

V5:

- Simpler runtime control loop; no scheduler freshness dependency for occupancy.  
[src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV5.kt:77-88]

V6:

- Requires scheduler health and tuning (`counter-refresh-every`, `counter-refresh-since`).
- Scheduler failure is logged and swallowed; system continues with stale counters.  
[src/main/kotlin/com/ratelimiter/slot/WindowCounterRefreshScheduler.kt:41-43]

Additional complexity note:

- A pre-provisioning scheduler is active at startup and daily for RL\_WNDW\_CT, driven by global settings (60 days default), which contradicts some README claims about no pre-provisioning in V5/V6.  
[src/main/kotlin/com/ratelimiter/slot/WindowPreProvisioningScheduler.kt:25-27,

src/main/kotlin/com/ratelimiter/slot/WindowPreProvisioningScheduler.kt:39-57,  
src/main/resources/application.yaml:135,  
src/main/kotlin/com/ratelimiter/slot/V5\_README.md:149-153,  
src/main/kotlin/com/ratelimiter/slot/V6\_README.md:177-183]

Verdict:

- Maintainability/ops simplicity winner: V5.

---

## C7) Multi-pod (~24) behavior

---

Shared strengths:

- Skip pointer is append-only with composite PK and MAX-by-order read.  
[src/main/kotlin/com/ratelimiter/repo/SkipPointerRepository.kt:31-39,  
src/main/kotlin/com/ratelimiter/repo/SkipPointerRepository.kt:51-63,  
src/main/kotlin/com/ratelimiter/db/Tables.kt:25-31]

Trade-off:

- V5 may incur row-update contention when many claims target same window.  
[src/main/kotlin/com/ratelimiter/repo/WindowSlotCounterRepository.kt:257-281]
- V6 removes hot-path counter updates but multiplies scheduler load with pod count.  
[src/main/kotlin/com/ratelimiter/slot/WindowCounterRefreshScheduler.kt:34-40]

Verdict:

- Tie (depends on measured concentration and DB headroom).

---

## C8) Quarkus per-JVM scheduler vs distributed Quartz singleton

---

Current config (3s refresh, since 6s):

- [src/main/resources/application.yaml:153-154]

Per-JVM Quarkus with N pods:

- Effective average refresh cadence can be  $\sim 3/N$  seconds if staggered.  
(Hypothesis; code does not guarantee perfect staggering.)  
[src/main/kotlin/com/ratelimiter/slot/WindowCounterRefreshScheduler.kt:34-40]

At 24 pods:

- Approx average refresh  $\sim 125\text{ms}$  ( $3/24$ ) if staggered.

Quartz singleton:

- Refresh cadence becomes 3s cluster-wide.

Implications:

- DB load drops sharply in singleton mode.
- Advisory occupancy staleness window grows  $\sim 24\times$  vs staggered 24-pod assumption.
- V6 selection quality and stale-repick frequency worsen; V5 unaffected by refresh model.

Verdict:

- Under Quartz singleton refresh, V5 is favored.

---

## C9) Other relevant parameters ( $\geq 5$ )

---

1) Failure modes:

- V6 has scheduler lag/failure mode; stale occupancy persists until refresh recovers.

[src/main/kotlin/com/ratelimiter/slot/WindowCounterRefreshScheduler.kt:41-43]

## 2) Consistency model:

- V5 counters updated inline per claim.  
[src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV5.kt:218-220]
- V6 counters are eventually reconciled by MERGE.  
[src/main/kotlin/com/ratelimiter/repo/WindowSlotCounterRepository.kt:233-251]

## 3) Retry/rollback behavior:

- V5: no rollback path for overshoot.  
[src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV5.kt:181-183]
- V6: guard reject -> exclude window -> re-pick loop; still no rollback.  
[src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV6.kt:214-217]

## 4) Hotspot contention:

- V5 has potential counter-row hotspot (update same row).  
[src/main/kotlin/com/ratelimiter/repo/WindowSlotCounterRepository.kt:257-281]
- V6 removes that from hot path.  
[src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV6.kt:228-241]

## 5) Resilience to scheduler lag:

- V6 sensitive to refresh interval/lookback tuning.  
[src/main/resources/application.yaml:153-154]

## 6) Idempotency under race:

- Same strong mechanism in both via unique eventId and duplicate re-read.  
[src/main/kotlin/com/ratelimiter/db/Tables.kt:36,  
src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV5.kt:212-216,  
src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV6.kt:243-247]

## 7) Observability:

- Scheduler has error logging but no explicit service-level metrics in analyzed files.  
[src/main/kotlin/com/ratelimiter/slot/WindowCounterRefreshScheduler.kt:31-43]

# D) Practical Worst-case Scenarios (>=3 per version)

## V5 worst-cases:

### 1) Concentrated-window contention:

- Multiple pods update the same WNDW\_STRT\_TS row; lock waits increase latency  
[src/main/kotlin/com/ratelimiter/repo/WindowSlotCounterRepository.kt:257-281]

### 2) MaxSlots overshoot under race:

- No claim-time hard cap check; multiple claimers can pass stale occupancy threshold.  
[src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV5.kt:117-123,  
src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV5.kt:181-183]

### 3) Horizon exhaustion at sustained overload:

- Throws SlotAssignmentException after phase search bounds are consumed.  
[src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV5.kt:90-96]

### 4) (Hypothesis) unaligned requestedTime lattice fragmentation:

- Practical global sharing weakens across streams.  
[src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV5.kt:234-237,  
src/main/kotlin/com/ratelimiter/api/SlotAssignmentV2Resource.kt:54]

## V6 worst-cases:

### 1) Scheduler fan-out DB pressure:

- 24-pod, 3s refresh drives repeated heavy MERGE/COUNT scans.  
[src/main/resources/application.yaml:153-154,  
src/main/kotlin/com/ratelimiter/repo/WindowSlotCounterRepository.kt:222-225]

## 2) Scheduler lag/failure stale advisory:

- Counters stale; soft guard preserves max-level filtering but repicks/extra COUNTs rise.  
[src/main/kotlin/com/ratelimiter/slot/WindowCounterRefreshScheduler.kt:41-43,  
src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV6.kt:214-217]

## 3) COUNT->INSERT race overshoot:

- Soft guard is non-atomic; concurrent passers can exceed maxSlots.  
[src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV6.kt:213-221,  
src/test/kotlin/com/ratelimiter/SlotAssignmentServiceV6Test.kt:568-575]

## 4) Quartz singleton freshness penalty:

- Refresh staleness increases, worsening advisory selection quality.  
[src/main/resources/application.yaml:153-154]

# =====

## E) Quarkus vs Quartz Recalculation (explicit)

# =====

### Config basis:

- counter-refresh-every = 3s
- counter-refresh-since = 6s  
[src/main/resources/application.yaml:153-154]

### At 500 TPS:

#### Case 1: Per-JVM Quarkus, 24 pods (staggered hypothesis)

- Avg effective interval  $\sim 3/24 = 0.125s$
- Events between effective refreshes  $\sim 500 * 0.125 = 62.5$
- Scheduler runs/sec cluster-wide  $\sim 8$
- Scheduler counted rows/sec  $\sim 8 * (45k..90k) = 360k..720k$   
[row estimate source: src/main/kotlin/com/ratelimiter/repo/WindowSlotCounterRepository.kt:222-225]

#### Case 2: Quartz cluster-singleton, 24 pods

- Effective interval = 3s
- Events between refreshes  $\sim 1500$
- Scheduler runs/sec cluster-wide  $\sim 0.333$
- Scheduler counted rows/sec  $\sim 15k..30k$

### Interpretation:

- Quartz singleton significantly lowers scheduler DB load.
- But it materially increases advisory staleness and stale-pick pressure.
- For V6 this is a direct quality/performance trade-off; V5 is much less sensitive.

# =====

## F) Final Recommendation (Corrected)

# =====

### Overall best default:

- V5 (for your current workload assumptions and concerns), with one condition:
  - confirm via measurement whether RL\_WNDW\_CT row lock waits are actually dominating.

### Version that stands out by parameter:

- Global rate limit behavior: Tie
- Proximity to requestedTime: Tie (slight V5)

- Correctness at target workloads: Tie
- Throughput under counter-row contention: V6
- Total DB load at high TPS (current per-pod scheduler): V5
- Ops simplicity: V5
- Multi-pod scaling: Tie (trade-off)
- Quartz singleton compatibility: V5

Decision matrix by priority:

- 1) Strict correctness-first:
  - Tie with caveats (neither is strict atomic max guarantee).
- 2) Throughput-first (if lock waits are proven bottleneck):
  - V6.
- 3) Ops-simplicity + predictable DB profile-first:
  - V5.

## =====

### G) Test Gaps and What to Benchmark Next

## =====

- 1) No true multi-pod benchmark for V5/V6 (tests are multi-thread single-JVM).
  - Add 20/24-pod load tests against shared Oracle.  
[src/test/kotlin/com/ratelimiter/SlotAssignmentServiceV5Test.kt:383-416,  
src/test/kotlin/com/ratelimiter/SlotAssignmentServiceV6Test.kt:383-417]
- 2) No explicit unaligned-requestedTime test.
  - Add randomized millis requestTime workload to verify global-sharing behavior.
- 3) No sustained 500 TPS for 1h benchmark for V5/V6.
  - Capture p50/p95/p99 latency, DB CPU, lock waits, and SlotAssignmentException rates.
- 4) No Quartz-singleton simulation for V6 refresh.
  - Compare per-pod refresh vs singleton refresh on same traffic.
- 5) Over-allocation assertions are loose in V6 tests ( $\leq \text{max} + \text{threadCount}$ ).
  - Add tighter p99 overshoot SLO under realistic pod/thread counts.  
[src/test/kotlin/com/ratelimiter/SlotAssignmentServiceV6Test.kt:568-575]
- 6) Scheduler outage/lag fault injection missing.
  - Pause refresh and measure stale-counter impact and recovery behavior.
- 7) Document/code drift checks are needed.
  - README claims should be aligned with current no-rollback code and active pre-provisioning scheduler.  
[src/main/kotlin/com/ratelimiter/slot/V5\_README.md:557-568,  
src/main/kotlin/com/ratelimiter/slot/SlotAssignmentServiceV5.kt:181-183,  
src/main/kotlin/com/ratelimiter/slot/WindowPreProvisioningScheduler.kt:25-57]

## =====

### H) Quant Note: V5 Counter Contention Math (explicit sensitivity)

## =====

Goal: quantify when V5 update contention is mild vs severe.

Definitions:

- $\lambda_{\text{row}} = 500 * p_{\text{hot}}$  (arrivals/sec to hottest window row)
- $h$  = lock-hold/service time per update txn (sec)
- $\mu = 1/h$
- $\rho = \lambda_{\text{row}} / \mu = \lambda_{\text{row}} * h$

Input hints:

- Config comments indicate typical rate-limiter DB hold times ~3-10ms.  
[src/main/resources/application.yaml:45-47]

If  $p_{\text{hot}} \sim 6.45\%$  (linear proximity weight with 30 windows, equal occupancy),  
then  $\lambda_{\text{row}} \sim 32.3/\text{s}$ .

- $h=3\text{ms} \Rightarrow \rho=0.097$
  - $h=5\text{ms} \Rightarrow \rho=0.161$
  - $h=10\text{ms} \Rightarrow \rho=0.323$
- $\Rightarrow$  usually manageable.

If traffic concentrates (example  $p_{\text{hot}}=25\%$ ):

- $\lambda_{\text{row}}=125/\text{s}$
  - $h=5\text{ms} \Rightarrow \rho=0.625$
- $\Rightarrow$  queueing starts to matter.

If  $p_{\text{hot}}=35\%$ :

- $\lambda_{\text{row}}=175/\text{s}$
  - $h=5\text{ms} \Rightarrow \rho=0.875$
- $\Rightarrow$  significant wait inflation.

Takeaway:

- V5 contention is not automatically severe.
- It becomes severe only when row concentration and/or hold time push  $\rho$  near 1.
- Therefore a measured decision is required; assumptions alone are insufficient.

## I) Final One-line Verdict

After applying the corrections you raised, the evidence supports:

- V5 as default choice for these workloads,
- V6 as conditional optimization only when measured V5 counter-row contention is proven to be the dominant bottleneck and scheduler overhead is acceptable.